

# CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

## Lecture 18: Caches I

Instructor: Justin Yokota

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches

# Agenda

- **Intro**
  - **Binary Prefixes**
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches

# Kilo, Mega, Giga, Tera, Peta, Exa, Zetta, Yotta

- Common-use prefixes (all SI, except K  $\neq$  k in SI)
- Confusing! Common usage of “kilobyte” means 1024 bytes, but the “correct” SI value is 1000 bytes
- Hard Disk manufacturers & Telecommunications are the only computing groups that use SI factors
  - What is advertised as a 1 TB drive actually holds about 90% of what you expect
  - A 1 Mbit/s connection transfers  $10^6$  bps.

# Chart

| Name  | Abbr | Factor                                       | SI size                                       |
|-------|------|----------------------------------------------|-----------------------------------------------|
| Kilo  | K    | $2^{10} = 1,024$                             | $10^3 = 1,000$                                |
| Mega  | M    | $2^{20} = 1,048,576$                         | $10^6 = 1,000,000$                            |
| Giga  | G    | $2^{30} = 1,073,741,824$                     | $10^9 = 1,000,000,000$                        |
| Tera  | T    | $2^{40} = 1,099,511,627,776$                 | $10^{12} = 1,000,000,000,000$                 |
| Peta  | P    | $2^{50} = 1,125,899,906,842,624$             | $10^{15} = 1,000,000,000,000,000$             |
| Exa   | E    | $2^{60} = 1,152,921,504,606,846,976$         | $10^{18} = 1,000,000,000,000,000,000$         |
| Zetta | Z    | $2^{70} = 1,180,591,620,717,411,303,424$     | $10^{21} = 1,000,000,000,000,000,000,000$     |
| Yotta | Y    | $2^{80} = 1,208,925,819,614,629,174,706,176$ | $10^{24} = 1,000,000,000,000,000,000,000,000$ |

# kibi, mebi, gibi, tebi, pebi, exbi, zebi, yobi

- IEC Standard Prefixes

| Name | Abbr | Factor                       |      |    |                                              |
|------|------|------------------------------|------|----|----------------------------------------------|
| kibi | Ki   | $2^{10} = 1,024$             | pebi | Pi | $2^{50} = 1,125,899,906,842,624$             |
| mebi | Mi   | $2^{20} = 1,048,576$         | exbi | Ei | $2^{60} = 1,152,921,504,606,846,976$         |
| gibi | Gi   | $2^{30} = 1,073,741,824$     | zebi | Zi | $2^{70} = 1,180,591,620,717,411,303,424$     |
| tebi | Ti   | $2^{40} = 1,099,511,627,776$ | yobi | Yi | $2^{80} = 1,208,925,819,614,629,174,706,176$ |

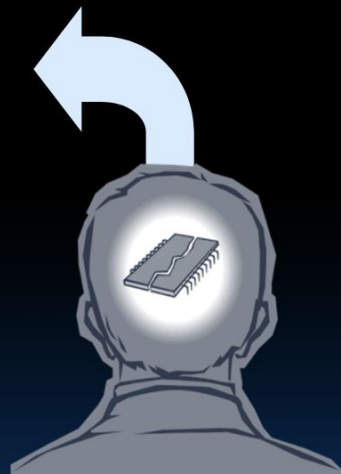
- International Electrotechnical Commission (IEC) in 1999 introduced these to specify binary quantities.
- Names come from shortened versions of the original SI prefixes (same pronunciation) and bi is short for “binary”, but pronounced “bee” :(
- Now SI prefixes only have their base-10 meaning and never have a base-2 meaning.

# The way to remember #s

- What is  $2^{34}$ ? How many bits to address (i.e., what's  $\text{ceil } \log_2 = \lg \text{ of } 2.5 \text{ TiB}$ )?

- Answer!  $2^{XY}$  means...

|                                       |                       |
|---------------------------------------|-----------------------|
| X=0 $\Rightarrow$ ---                 | Y=0 $\Rightarrow$ 1   |
| X=1 $\Rightarrow$ kibi $\sim 10^3$    | Y=1 $\Rightarrow$ 2   |
| X=2 $\Rightarrow$ mebi $\sim 10^6$    | Y=2 $\Rightarrow$ 4   |
| X=3 $\Rightarrow$ gibi $\sim 10^9$    | Y=3 $\Rightarrow$ 8   |
| X=4 $\Rightarrow$ tebi $\sim 10^{12}$ | Y=4 $\Rightarrow$ 16  |
| X=5 $\Rightarrow$ pebi $\sim 10^{15}$ | Y=5 $\Rightarrow$ 32  |
| X=6 $\Rightarrow$ exbi $\sim 10^{18}$ | Y=6 $\Rightarrow$ 64  |
| X=7 $\Rightarrow$ zebi $\sim 10^{21}$ | Y=7 $\Rightarrow$ 128 |
| X=8 $\Rightarrow$ yobi $\sim 10^{24}$ | Y=8 $\Rightarrow$ 256 |
|                                       | Y=9 $\Rightarrow$ 512 |



**MEMORIZE!**

# Agenda

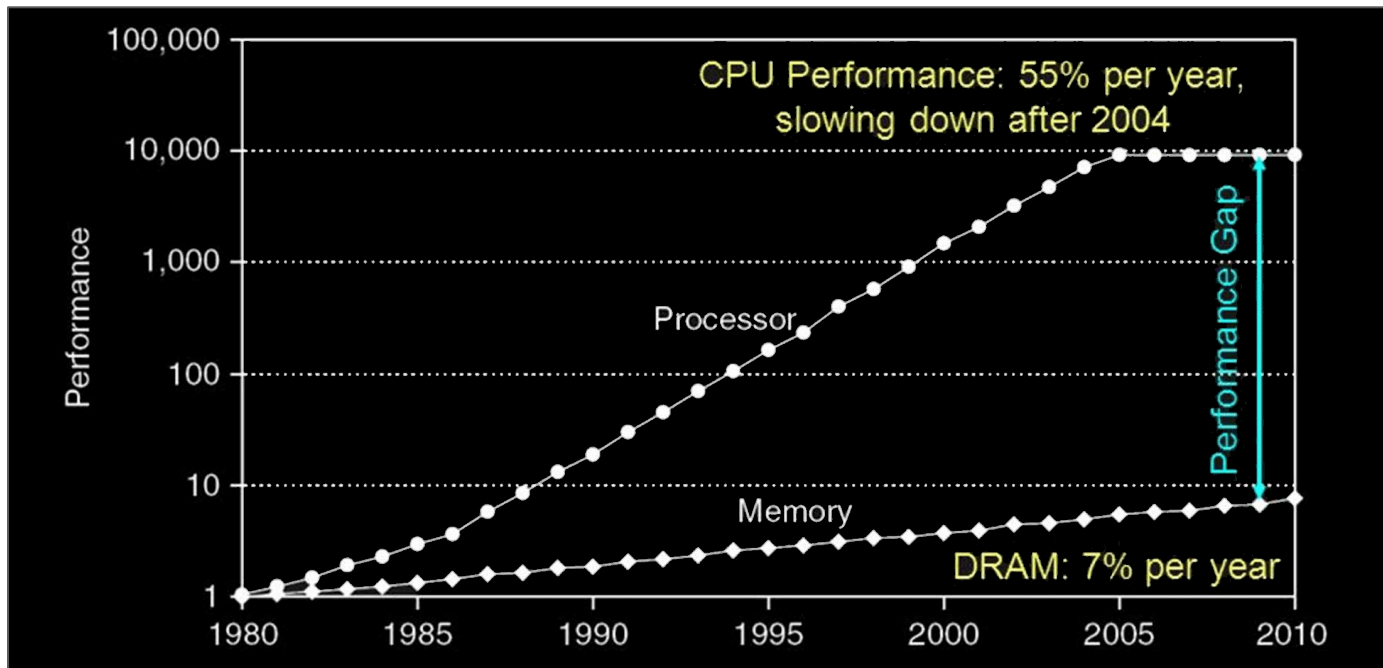
- **Intro**
  - Binary Prefixes
  - **Memory Hierarchy**
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches



# Moore's Law

- Technological improvement is exponential
  - New tech yields better production methods, which yields better tech (up to a physical limit)
- Moore's Law: processor power used to double every ~18 months
  - Similar exponential growth exists in other computer aspects (e.g. memory capacity, memory access time, etc.)
- However, exponential growth is highly dependent on the base
  - If two things grow at different exponential rates, then the difference between the two grow exponentially...

# Processor speed vs Memory Access Time



1980 microprocessor executes ~one instruction in same time as DRAM access  
2020 microprocessor executes ~1000 instructions in same time as DRAM access

Slow DRAM access has disastrous impact on CPU performance!

# Processor Speed vs Memory Access Time

- In our datapath, a single RISC-V instruction takes  $<1$  nanoseconds to complete **as long as it stays within the CPU (only looks at registers)**
  - One instruction completes in one clock cycle
- Memory operations (loads and stores) need to access main memory, which adds significant overhead
  - Light travels 1 foot per nanosecond. If our computer is larger than 1 foot, it is physically impossible to do a memory access in the same time as an instruction.

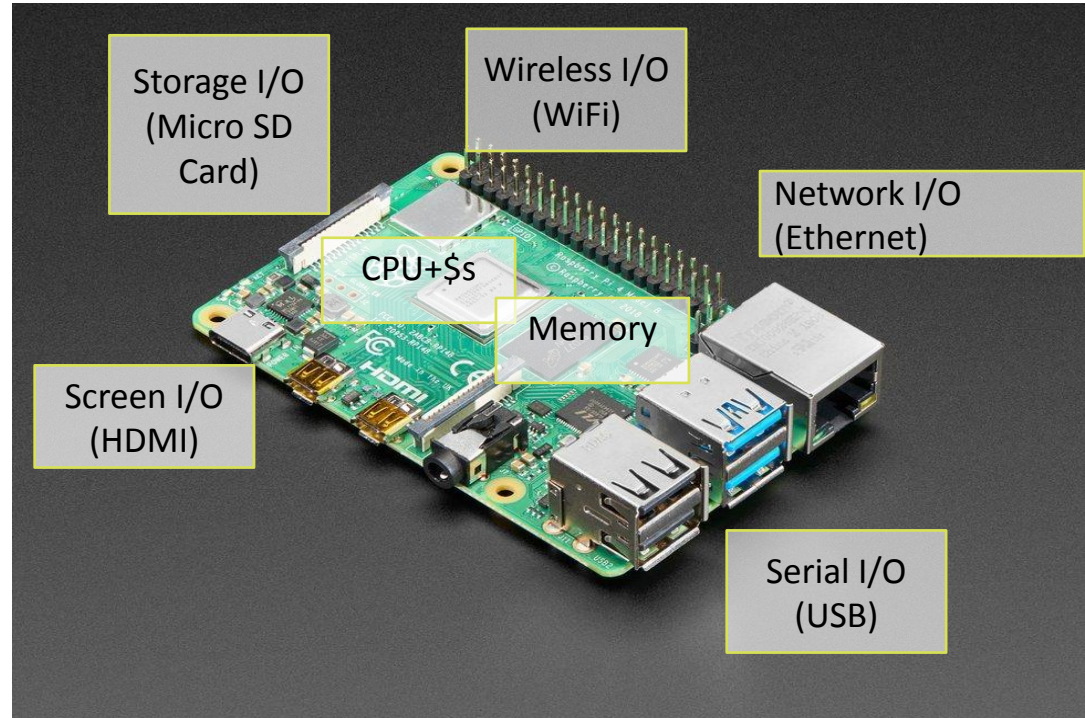
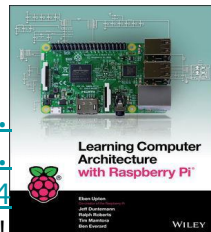
# Raspberry Pi (\$35) and I/O ports

The Raspberry Pi is a low-cost computer.

- Motherboard with CPU, I/O, caches, etc., soldered on
- **Cache Memory** on processor ( “\$” stands for cache)
- **Main Memory**
- **Disk (Secondary Memory)**

<https://onlinelibrary.wiley.com/doi/book/10.1002/9781119415534>

Free w/UCB login!

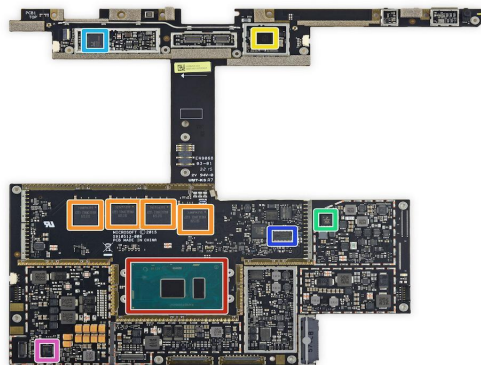


# Main Memory is DRAM

- Dynamic Random Access Memory:
  - Latency to access first word:  $\sim 10\text{ns}$  ( $\sim 30\text{--}40$  processor cycles), each successive ( $0.5\text{ns} - 1\text{ns}$ )
  - Each access brings 64 bits, supports 'bursts'
  - $\$3/\text{GiB}$
- Data is impermanent:
  - **Dynamic**: capacitors store bits, so needs periodic refresh to maintain charge
  - **Volatile**: when power is removed, loses data.
- Contrast with SRAM (for caches):
  - Static (no capacitors) but still volatile
  - Faster ( $0.5\text{ ns}$ )/more expensive/lower density



Desktop/server  
DIMM



MS Surface Book

# Storage / “Disk” / Secondary Memory

Attached as a peripheral I/O device. **Non-volatile**

- Solid-State Drive (SSD)

- Access: 40-100 $\mu$ s  
(~100k proc. cycles)
- \$0.05-0.5/GB
- Usually flash memory



- Hard Disk Drive (HDD)

- Access: <5-10ms  
(10-20M proc. cycles)
- \$0.01-0.1/GB
- Mechanical



# Operation Time Costs

- As a general rule of thumb, the runtime cost of operations gets broken down as follows:
- File operations
  - Extremely slow (retrieving from disk, so it takes a long time)
  - Also includes prints and other I/O, so it's a good idea to avoid printing lots of data when testing runtime.
- Memory operations
  - ~100x faster than file operations (accessing RAM)
  - Can be improved through caching, but memory operations still take 10-100 clock cycles
- Branches and Jumps
  - ~5 cycles due to control hazards
  - Can be improved through loop unrolling
- Arithmetic operations
  - Fastest operations (1 cycle ideally, though some operations take longer)

# Why are large memories slow? Library Analogy

- You want to write a report using library books
- Our desk at home is the CPU; we can only do work while at home
- The library is the DMEM; it contains all the information we need to write our report
  - Also, notebooks are also stuck at the library, so we store all our heap memory in the library
- Currently: When we need to read a memory address, we call the library and ask for them to find our data.
- The librarians search the library, finds the particular book, and finds the line within the book we need.
- The librarians memorize the line, put the book back, and call us back with the line of data we need



# Why are large memories slow? Library Analogy

- Time to find a book in a large library
  - Search a large card catalog – (mapping title/author to index number)
  - Round-trip time to walk to the stacks and retrieve the desired book
- Larger libraries worsen both delays
- Electronic memories have same issue, plus the technologies used to store data slow down as density increases (e.g., SRAM vs. DRAM vs. Disk)
- If registers are what you store in your brain, then proportionally:
  - Main memory (RAM) is a library on campus (if you walk there)
  - Disk memory is a library in Los Angeles (if you walk there)
  - Tape memory is a library on the moon (if you walk there)

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- **Intro to Caches**
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches

# Library Analogy: Ways to speed things up?

- Option 1: While you're waiting for the data you need to do your English paper, switch to a different task and do your math homework
- Option 2: Plan ahead and ask for the data ahead of time. By the time you need that data, the librarians are ready to call you back
- Option 3: Instead of just getting one line of data at a time, borrow some of the library books and store them at home so you don't need to go all the way to the library

# Option 1: Hyperthreading

- Option 1: While you're waiting for the data you need to do your English paper, switch to a different task and do your math homework
- In a computer: While waiting for a DMEM access on one program, context switch and run another program
  - Context switching is expensive, so needs specialized hardware to get speedup
- Intel's version of this technology is called hyperthreading
- Done at the OS level, and doesn't speed up an individual task

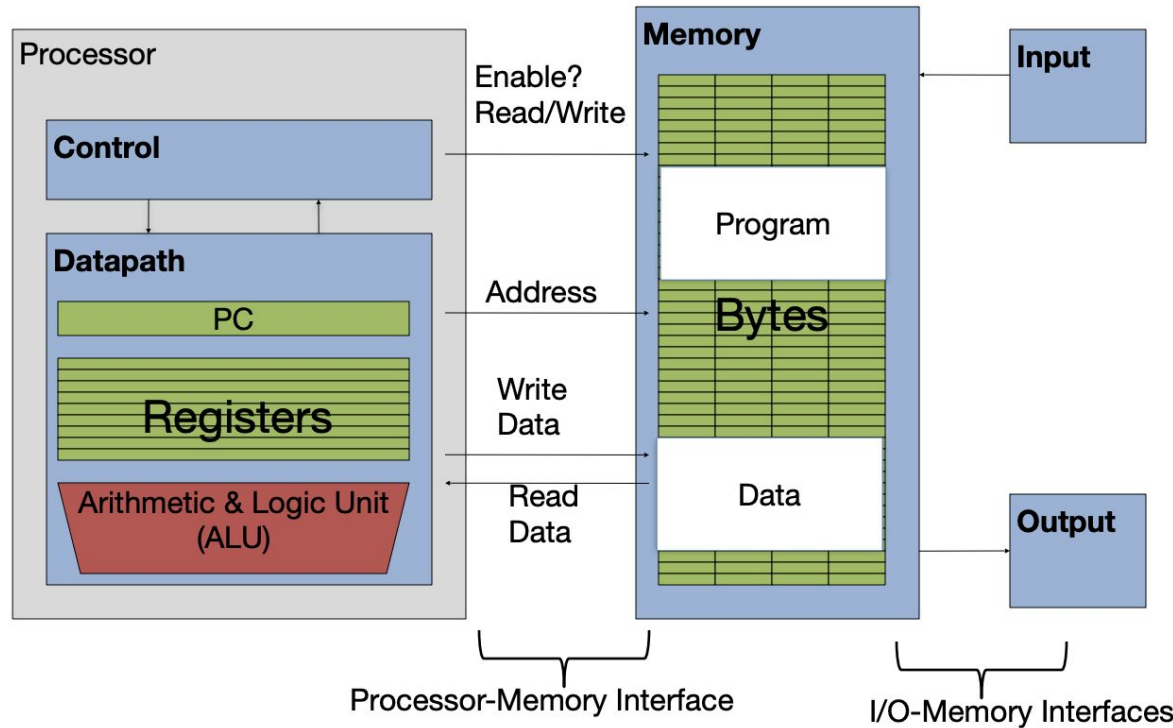
# Option 2: Prefetching

- Option 2: Plan ahead and ask for the data ahead of time. By the time you need that data, the librarians are ready to call you back
- In a computer: Request data several cycles before you actually use the data
- Can be done manually with prefetching commands in C, but this is out of scope for this course.
- Software-based, so flexible, but also limited.

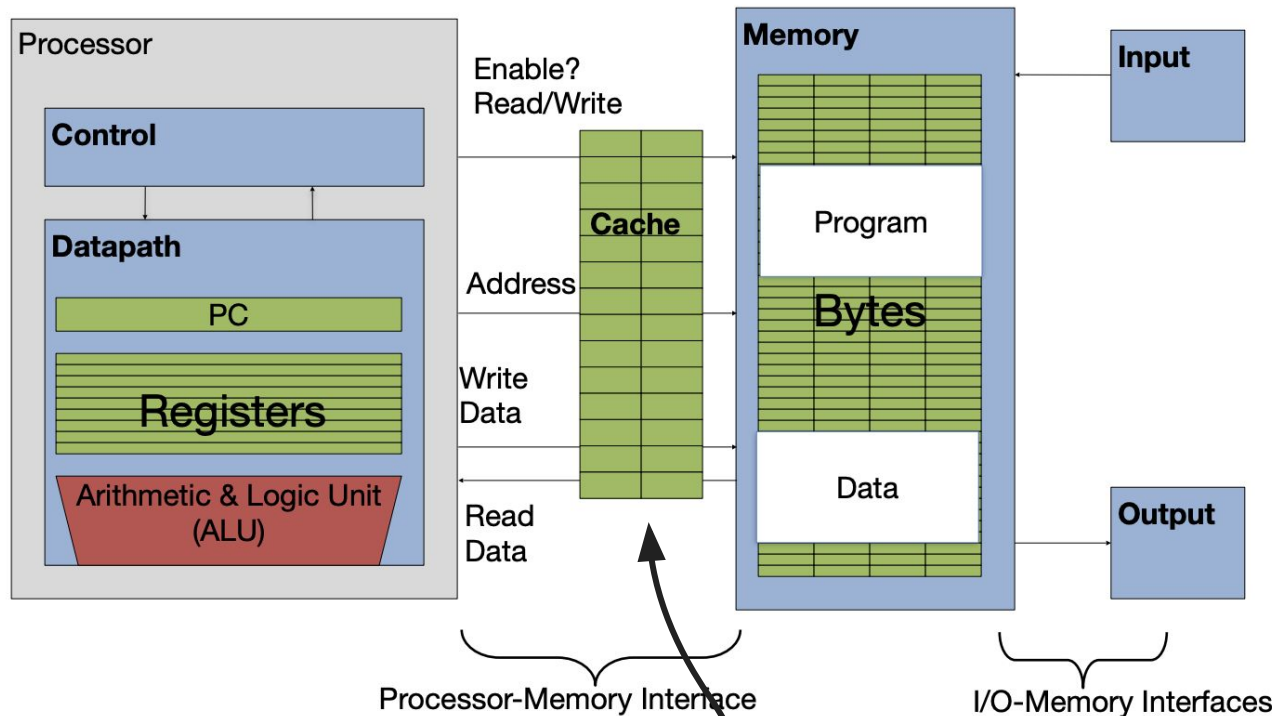
# Option 3: Caching

- Option 3: Instead of just getting one line of data at a time, borrow some of the library books and store them at home so you don't need to go all the way to the library
- Main idea: If you use a book once, chances are you'll use it again.
- Solution: Get a bookshelf at home that you can use to keep a few books, and "borrow" the book from the library when you read it.
- If we need another book, go back to the library and borrow a new one
  - But don't return earlier books since you might need them, until your bookshelf is too full to carry more books
- You hope this collection of ~10 books at home enough to write report, despite 10 being only 0.00001% of books in UC Berkeley libraries

# Computer Without Cache



# Computer With Cache



**Lines** (i.e., blocks) of data are copied from memory to the cache.

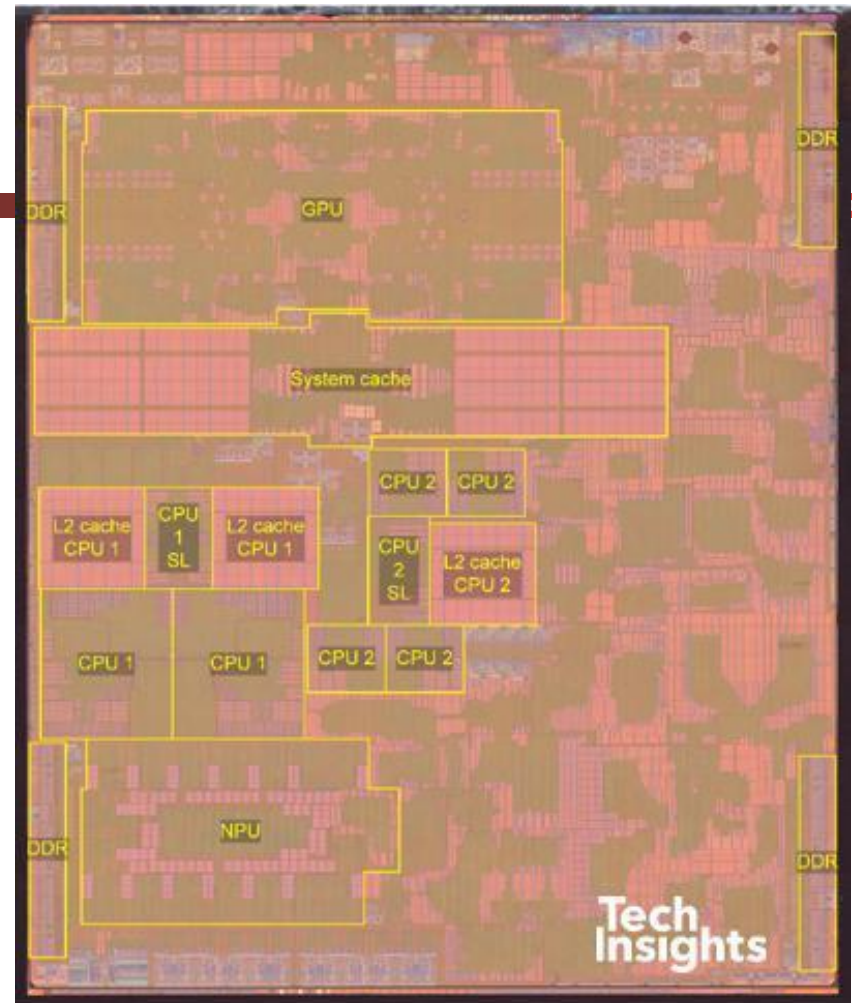


# Memory Cache

CS 61C

- Mismatch between processor and memory speeds leads us to add a new level: The “**memory cache**”, or **cache** for short.
  - Usually on the same chip as the CPU.
  - **Faster but more expensive** than DRAM memory.
- The cache is a copy of a subset of main memory.

Apple A14 Bionic  
TechInsights [[source](#)]



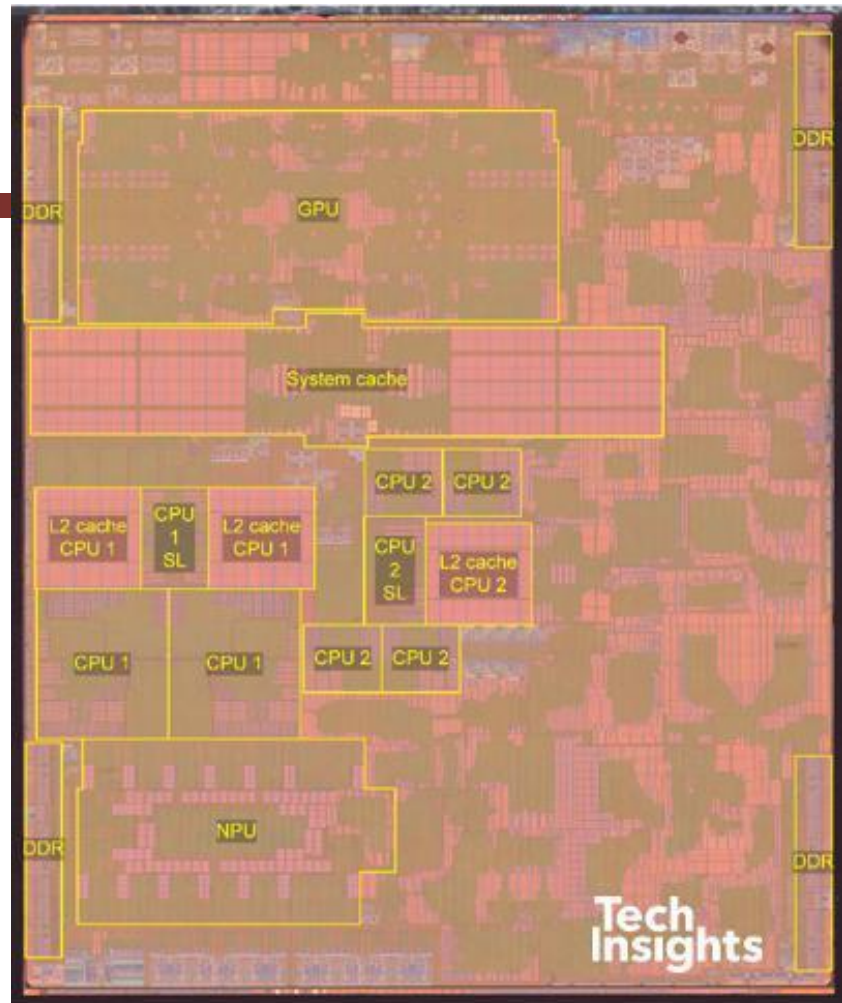
026

# Memory Cache

CS 61C

- Abbreviation: “\$” = cache
- L1 cache is usually directly embedded on the CPU cache.
  - Often embedded into two parts, L1i (instruction) and L1d (data).
  - IMEM and DMEM are the two caches on the **CPU Datapath!**
- L2 cache located on integrated circuit, often adjacent to CPU
- “System Level Cache” (out of scope)

Apple A14 Bionic  
TechInsights [[source](#)]



# Locality

- Caching is only useful if you can predict what memory a program will access soon.
  - If a program uses completely random memory accesses, caches are guaranteed to (on average) slow down your access time!
- Can't use data about the program itself to decide this (that's prefetching)
- Can we predict future memory accesses given the most recent memory accesses?
  - Useful to look at **benchmarks** to find common memory access patterns
- For these lectures, we'll use Matrix Multiplication as our benchmark

# What patterns do we see in MatMul?

```
// a = i*j matrix, b = j*k matrix, c = result, i*k matrix
matrix* matmul(matrix* a, matrix* b, matrix* c, int i_max, int
j_max, int k_max) {
    for(int i = 0; i < i_max; i++) {
        for(int k = 0; k < k_max; k++) {
            int result = 0;
            for(int j = 0; j < j_max; j++) {
                result += a[i*j_max+j]*b[j*k_max+k];
            }
            c[i*k_max+k] = result;
        }
    }
}
```

# Locality

- Temporal Locality

- If a memory location is referenced then it will tend to be referenced again soon
  - Ex. Matmul reads the same matrix multiple times in quick succession
- Keep most recently accessed data items in our cache

- Spatial Locality

- If a memory location is referenced, the locations with nearby addresses will tend to be referenced soon
  - Ex. Matmul accesses memory in a contiguous sequence of addresses
- Move blocks consisting of contiguous words instead of just one word at a time

# Blocks

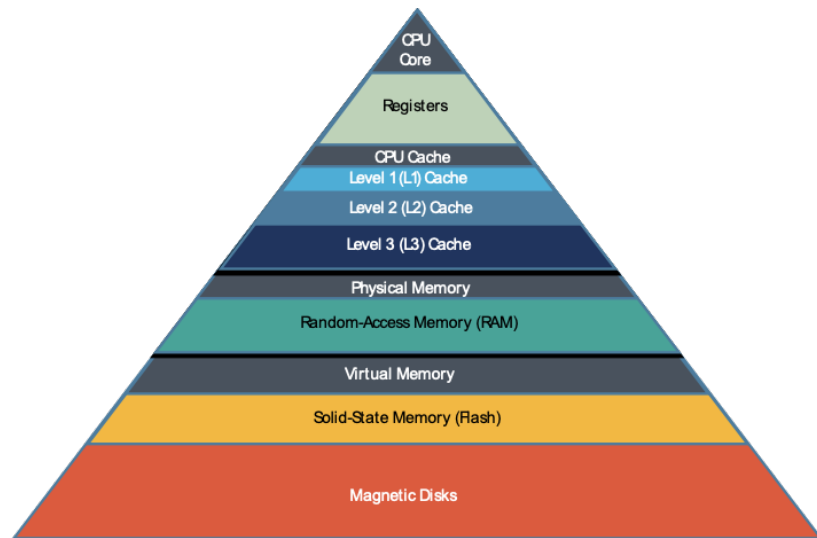
- Move blocks consisting of contiguous words instead of just one word at a time
- Divide all memory into "blocks" of size some power of 2
  - Analogy: Each memory address is a line in a book, each block is a book
- Blocks will be assigned a number, called their tag
- Ex. If we have blocks of size  $4096 = 2^{12}$ :
  - Block 0 will contain data from `0x0000 0000` to `0x0000 0FFF`
  - Block 1 will contain data from `0x0000 1000` to `0x0000 1FFF`
  - Block `0xABCD E` will contain data from `0xABCD E000` to `0xABCD EFFF`
- A memory address can be divided into a tag, and an offset
  - Ex. `0xDEADBEEF` with a block size of 4096
  - This address is in Block `0xDEADB`, and is the `0xEEF`th byte in that block

# Memory Hierarchy Basis

- Caches are an intermediate memory level between fastest and most expensive memory (registers) and the slower components (DRAM)
  - DRAM can be considered a "cache" for disk in a way!
- Cache contains copies of data in memory that are being used.
- Memory contains copies of data on disk that are being used.
- Caches work on the principles of temporal and spatial locality.
  - Temporal locality (locality in time): If we use it now, chances are we'll want to use it again soon.
  - Spatial locality (locality in space): If we use a piece of memory, chances are we'll use the neighboring pieces soon.

# How is the hierarchy managed?

- registers « memory
  - By compiler (or assembly level programmer)
- cache « main memory
  - By the cache controller hardware
- main memory « disks (secondary storage)
  - By the operating system (virtual memory)
  - Virtual to physical address mapping assisted by the hardware ('translation lookaside buffer' or TLB)
  - By the programmer (files)





# Agenda

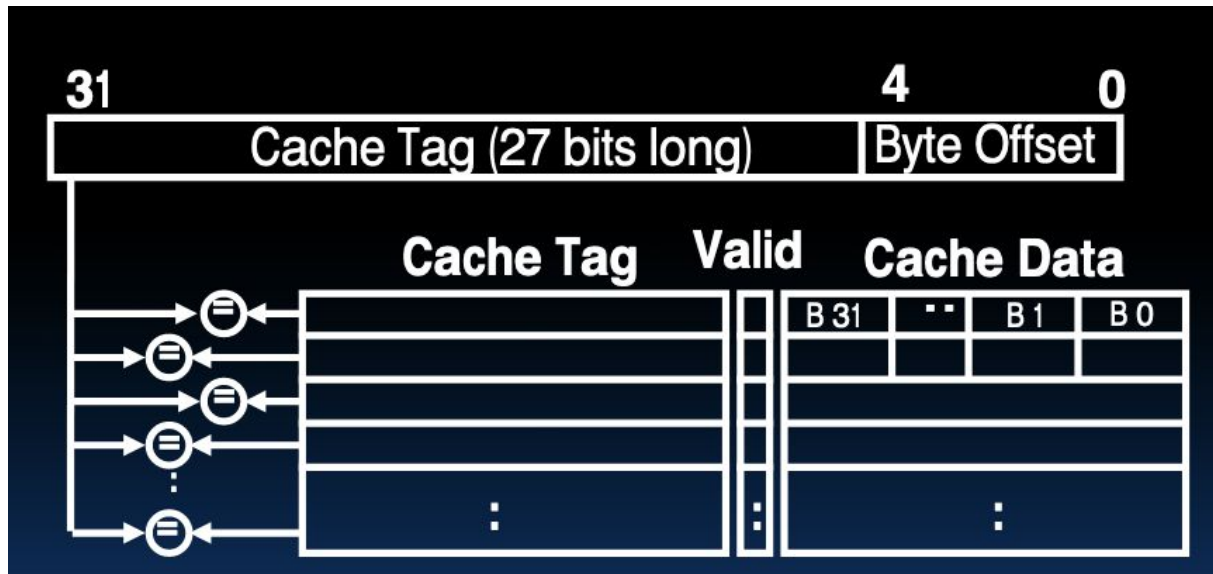
- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- **Fully Associative (FA) Caches**
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches

# Fully Associative Cache (1/4)

- A fully associative cache is parametrized on two aspects: The block size, and the number of blocks that can be stored in the cache.
- When a memory access occurs:
  - Step 1: Check if the cache contains the data needed. If so, return the data.
  - Step 2: If the data is not present in the cache, load the block of memory into the cache
  - Step 3: Return the data
- Library analogy: You buy a bookshelf that can hold N books
  - Step 1: Check if your bookshelf has the book you need. If so, get the data
  - Step 2: If you don't have the needed book, go to the library to borrow it
  - Step 3: Return the data.

# Fully Associative Cache (2/4)

- Fully Associative Cache (with 32 B block)
  - Tag identifies what block is stored
  - Valid is a 1-bit value that differentiates actual data from garbage
    - In any hardware system, each bit is always set to something.



# Fully Associative Cache (3/4): Example Memory Access

- Let's say we have a cache with 4 byte blocks and 4 blocks storage, and let's assume that we're working with a system that uses 10-bit addresses
- To access address 0x3C1:
  - Split into tag and offset:
    - 0x3C1 == 0b1111 0000 01
  - Tag matches a block in our cache, and the valid bit is on, so our data is in the cache
  - We get the 1st byte of the block, which is 0xDE

| Tag         | Valid | Data                   |
|-------------|-------|------------------------|
| 0b1001 1001 | 0     | 0xDE 0xAD 0xBE<br>0xEF |
| 0b1011 0011 | 0     | 0x01 0x23 0x45<br>0x67 |
| 0b0110 1011 | 1     | 0xAB 0xAD 0xCA<br>0xFE |
| 0b1111 0000 | 1     | 0xAC 0xDE 0xFF<br>0x61 |

# Fully Associative Cache (3/4): Example Memory Access

- Let's say we have a cache with 4 byte blocks and 4 blocks storage, and let's assume that we're working with a system that uses 10-bit addresses
- To access address 0x266:
  - Split into tag and offset:
    - $0x266 == 0b1001\ 1001\ 10$
  - Tag matches a block in our cache, but the valid bit is off, so our data is not in our cache
  - We need to go to main memory to load the block.

| Tag         | Valid | Data                   |
|-------------|-------|------------------------|
| 0b1001 1001 | 0     | 0xDE 0xAD 0xBE<br>0xEF |
| 0b1011 0011 | 0     | 0x01 0x23 0x45<br>0x67 |
| 0b0110 1011 | 1     | 0xAB 0xAD 0xCA<br>0xFE |
| 0b1111 0000 | 1     | 0xAC 0xDE 0xFF<br>0x61 |

# Fully Associative Cache (4/4): Cache Terminology

- When reading memory, 3 things can happen:
  - **Cache Hit:**  
Cache block is valid and contains proper address, so read desired word
  - **Cache Miss:**  
Nothing in cache in appropriate block, so fetch from memory and replace an invalid block
  - Cache Miss with **Eviction:**  
Cache Miss, but the cache is already full of valid data, so we need to remove one block from the cache before we can load the new block
    - How to choose which block to evict is known as the **Eviction Policy** - more on this soon
- A cache is considered **Cold** if it doesn't have valid data in it
  - Often when starting a computer initially, or when recovering from a context switch
- A cache is considered **Hot** if it has valid data, and there's a high ratio of Cache Hits

# Cache Timing

- Compare the naive approach (just going to the library) to a cache. With a cache:
  - We need to check our cache for data, so this takes a small amount of runtime
  - There's a chance that we don't need to go to the library at all, because our data is in the cache, so this saves a lot of runtime **sometimes**. The hotter the cache, the more likely we save runtime.
- Since the cache is a hardware component, we are forced to use it for all programs
  - Therefore, if our program isn't cache-efficient, this can slow down runtime!
  - No free lunch theorem: For a random sequence of memory operations, a cache will on average slow down total runtime, unless the cache contains all data in main memory.
  - We get speedup only because we never use truly random memory accesses
- In practice, cache hits are around 10 cycles, while cache misses are around 100-1000 cycles, so we get a big improvement from every cache hit.

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- **Matrix Multiply Example**
- Eviction Policy
- Write-Back/Write-Through Caches
- Analyzing Caches



# Caching Example: Matrix Multiply

- Assumptions:
  - Elements are 4 bytes
    - But only the bottom byte will be shown in the cache for space
  - Block size is 16 bytes (one row of this matrix)
  - Matrix **A** stored at 0x1000
  - Matrix **B** stored at 0x2000
  - Matrix **C** stored at 0x3000
  - No other memory used
  - Caches start cold
- Let's see how this works on a fully associative cache with 4 blocks

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
|    |    |    |    | B  |    |    |    |
|    |    |    |    | 17 | 18 | 19 | 20 |
|    |    |    |    | 21 | 22 | 23 | 24 |
|    |    |    |    | 25 | 26 | 27 | 28 |
|    |    |    |    | 29 | 30 | 31 | 32 |
| A  |    |    |    | C  |    |    |    |
| 1  | 2  | 3  | 4  |    |    |    |    |
| 5  | 6  | 7  | 8  |    |    |    |    |
| 9  | 10 | 11 | 12 |    |    |    |    |
| 13 | 14 | 15 | 16 |    |    |    |    |

# Caching Example: Fully Associative

- Start: Cache starts cold
  - Note: random data in the cache, but all valid bits are zero
- 0 misses, 0 hits

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 0     | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0     | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative

- Access  $A[0] = 0x1000$ 
  - Miss: Data isn't valid, so pull from main memory
- 1 miss, 0 hits

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 0     | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0     | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative

- Access B[0] = 0x2000
  - Miss: Data isn't valid, so pull from main memory
- 2 miss, 0 hits

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 1     | 0x01 0x02 0x03 0x04 |
| 0x733 | 0     | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0xB5 0x81 0x67 0x3F |

|   |    |    |    |    |
|---|----|----|----|----|
| B | 17 | 18 | 19 | 20 |
|   | 21 | 22 | 23 | 24 |
|   | 25 | 26 | 27 | 28 |
|   | 29 | 30 | 31 | 32 |

|    |    |    |    |
|----|----|----|----|
| A  |    |    |    |
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

|   |  |  |  |
|---|--|--|--|
| C |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |

# Caching Example: Fully Associative

- Access A[1]: 0x1004
  - Hit
- Access B[4]: 0x2010
  - Miss
- 3 Misses, 1 Hit

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 1     | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 0x11 0x12 0x13 0x14 |
| 0x156 | 0     | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative

- Access A[2]: 0x1008
  - Hit
- Access B[8]: 0x2020
  - Miss
- 4 Misses, 2 Hit

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 1     | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 0x11 0x12 0x13 0x14 |
| 0x201 | 1     | 0x15 0x16 0x17 0x18 |
| 0x4E9 | 0     | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative

- Access A[3]: 0x100C
  - Hit
- Access B[12]: 0x2030
  - Miss
- Problem: The cache is full
  - Which block do we evict to make space?

| Tag   | Valid | Data                |
|-------|-------|---------------------|
| 0x100 | 1     | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 0x11 0x12 0x13 0x14 |
| 0x201 | 1     | 0x15 0x16 0x17 0x18 |
| 0x202 | 1     | 0x19 0x1A 0x1B 0x1C |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- **Eviction Policy**
- Write-Back/Write-Through Caches
- Analyzing Caches



# Eviction Policies

- Our cache is necessarily smaller than main memory, so we will eventually reach a point where our cache is full, and we need to evict some block.
- Ideally, choose an eviction policy that evicts the least useful data. We have several options:
  - LRU
  - MRU
  - FIFO
  - LIFO
  - Random

# Eviction Policies

- Least Recently Used (LRU): evict the block in the set that is the oldest previous access.
  - Pro: temporal locality
    - recent past use implies likely future use: in fact, this is a very effective policy
  - Con: Requires complicated hardware and much time to keep track of this
- Most Recently Used (MRU): evict the block in the set that is the newest previous access.
- First-In-First-Out (FIFO): evict the oldest block in the set (queue).
- Last-In-First-Out (LIFO): evict the newest block in the set (stack).
  - Both FIFO and LIFO ignore order of accesses after/before the first/last access, but they serve as good approximations to LRU and MRU without adding too much excess hardware
- Random: randomly selects a block to evict
  - Works surprisingly okay, especially given a low temporal locality workload
  - Does not perform exceedingly well, but also not exceedingly poorly

# Eviction Policies

- Our cache is necessarily smaller than main memory, so we will eventually reach a point where our cache is full, and we need to evict some block.
- Ideally, choose an eviction policy that evicts the least useful data. We have several options
- For now: Let's restart our example assuming we use a LRU cache

# Caching Example: Fully Associative with LRU

- Start: Cache starts cold
  - Note: New LRU field containing which block accessed first
    - Lower number = more recent for today
- 0 misses, 0 hits

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 0     | 0   | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0     | 0   | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

|   |    |    |    |    |
|---|----|----|----|----|
| B | 17 | 18 | 19 | 20 |
|   | 21 | 22 | 23 | 24 |
|   | 25 | 26 | 27 | 28 |
|   | 29 | 30 | 31 | 32 |
| A |    |    |    |    |
|   | 1  | 2  | 3  | 4  |
|   | 5  | 6  | 7  | 8  |
|   | 9  | 10 | 11 | 12 |
|   | 13 | 14 | 15 | 16 |
| C |    |    |    |    |
|   |    |    |    |    |
|   |    |    |    |    |
|   |    |    |    |    |
|   |    |    |    |    |

# Caching Example: Fully Associative with LRU

- Access  $A[0] = 0x1000$ 
  - Miss

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 0     | 0   | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0     | 0   | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative with LRU

- Access B[0] = 0x2000
  - Miss

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 1   | 0x01 0x02 0x03 0x04 |
| 0x733 | 0     | 0   | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative with LRU

- Access A[1]: 0x1004
  - Hit
- Access B[4]: 0x2010
  - Miss

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 2   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 1   | 0x11 0x12 0x13 0x14 |
| 0x156 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative with LRU

- Access A[2]: 0x1008
  - Hit
- Access B[8]: 0x2020
  - Miss
- Access A[3]: 0x100C
  - Hit

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 2   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 3   | 0x11 0x12 0x13 0x14 |
| 0x201 | 1     | 1   | 0x15 0x16 0x17 0x18 |
| 0x4E9 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

|   |    |    |    |    |
|---|----|----|----|----|
| B | 17 | 18 | 19 | 20 |
|   | 21 | 22 | 23 | 24 |
|   | 25 | 26 | 27 | 28 |
|   | 29 | 30 | 31 | 32 |

|    |    |    |    |
|----|----|----|----|
| A  |    |    |    |
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

|   |  |  |  |
|---|--|--|--|
| C |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |



# Caching Example: Fully Associative with LRU

- Access B[12]: 0x2030
  - Miss
  - Cache is full, so evict the block with tag 0x200

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 2   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1     | 4   | 0x11 0x12 0x13 0x14 |
| 0x201 | 1     | 3   | 0x15 0x16 0x17 0x18 |
| 0x202 | 1     | 1   | 0x19 0x1A 0x1B 0x1C |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative with LRU

- Note: At this point, we've gotten a lot of misses, especially since we just evicted 0x2004, even though we never used it.
  - If we keep going, 60 hits and 68 misses from A and B alone
  - Can be resolved if we transpose B before running this code

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 3   | 0x01 0x02 0x03 0x04 |
| 0x203 | 1     | 1   | 0x1D 0x1E 0x1F 0x20 |
| 0x201 | 1     | 4   | 0x15 0x16 0x17 0x18 |
| 0x202 | 1     | 2   | 0x19 0x1A 0x1B 0x1C |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Fully Associative with LRU

- Write to C[0] = 0x3000
  - Miss, so retrieve from memory
- Problem: How to keep the cache and main memory consistent when you write data?

| Tag   | Valid | LRU | Data                |
|-------|-------|-----|---------------------|
| 0x100 | 1     | 3   | 0x01 0x02 0x03 0x04 |
| 0x203 | 1     | 1   | 0x1D 0x1E 0x1F 0x20 |
| 0x201 | 1     | 4   | 0x15 0x16 0x17 0x18 |
| 0x202 | 1     | 2   | 0x19 0x1A 0x1B 0x1C |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

B

|    |    |    |    |
|----|----|----|----|
| 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 |
| 29 | 30 | 31 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- **Write-Back/Write-Through Caches**
- Analyzing Caches

# Write Policy

- Write-through
  - When a write occurs, update the data both in the cache and in main memory
  - Slows down writes a lot, since we need to write to main memory every time
- Write-back
  - When a write occurs, only update the data in the cache
  - Purposefully let main memory go "stale", and rely on the cache as the main source of truth
  - When we later evict the block, write all changes to main memory at the same time
  - Needs a "dirty" bit to signify that the block has to be written to memory
  - Also means we need to be careful that we don't access main memory before we write back our data-more on this Friday
  - Much faster than write-through, and also extends computer lifespan, since RAM tends to degrade with each write.
- Let's rerun the simulation with a write-back cache, and after we transpose B.

# Caching Example: Write-Back Fully Associative with LRU

- Start: Cache starts cold
  - Note: New Dirty field
  - Note: Matrix B has been transposed into Bt to optimize cache efficiency
- 0 misses, 0 hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 0 | 0     | 0   | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0 | 0     | 0   | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- Access  $A[0] = 0x1000$ 
  - Miss
- Access  $Bt[0] = 0x2000$ 
  - Miss

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 0 | 0     | 0   | 0xBF 0x16 0x88 0x2B |
| 0x733 | 0 | 0     | 0   | 0x3B 0x18 0xF1 0xB3 |
| 0x156 | 0 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- A[1]: Hit Bt[1]: Hit
- A[2]: Hit Bt[2]: Hit
- A[3]: Hit Bt[3]: Hit
- 6 Hits, 2 Misses!

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 2   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1 | 0     | 1   | 0x11 0x15 0x19 0x1D |
| 0x156 | 0 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |



# Caching Example: Write-Back Fully Associative with LRU

- Write to C[0] = 0x3000
  - Miss + Write, so set dirty bit
- Main memory not updated yet

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 2   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1 | 0     | 1   | 0x11 0x15 0x19 0x1D |
| 0x156 | 0 | 0     | 0   | 0xE6 0x57 0x49 0xEE |
| 0x4E9 | 0 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- A[0]: Hit Bt[4]: Miss ... A[3]: Hit Bt[7]: Hit. Write C[1]: Hit
- 8 Hits, 1 Miss here
- Totals: 4 Misses, 14 Hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 3   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1 | 0     | 2   | 0x11 0x15 0x19 0x1D |
| 0x300 | 1 | 1     | 1   | 0xFA 0x?? 0x?? 0x?? |
| 0x4E9 | 0 | 0     | 0   | 0xB5 0x81 0x67 0x3F |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- A[0]: Hit Bt[8]: Miss, evict block 0x200, ... , Write C[2]: Hit
- 8 Hits, 1 Miss here
- Totals: 5 Misses, 22 Hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 3   | 0x01 0x02 0x03 0x04 |
| 0x200 | 1 | 0     | 4   | 0x11 0x15 0x19 0x1D |
| 0x300 | 1 | 1     | 1   | 0xFA 0x04 0x?? 0x?? |
| 0x201 | 1 | 0     | 2   | 0x12 0x16 0x1A 0x1E |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- Compute C[3]: 8 hits, 1 miss, block 0x201 evicted
- A[4]: Miss, Block 0x202 evicted
- 8 Hits, 2 Misses here
- Totals: 7 Misses, 30 Hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 3   | 0x01 0x02 0x03 0x04 |
| 0x202 | 1 | 0     | 2   | 0x13 0x17 0x1B 0x1F |
| 0x300 | 1 | 1     | 1   | 0xFA 0x04 0x0E 0x?? |
| 0x201 | 1 | 0     | 4   | 0x12 0x16 0x1A 0x1E |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- Bt[0]: Miss, evict block 0x100
- A[5]-Bt[3]: 6 hits
- Total: 8 misses, 36 hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x100 | 1 | 0     | 4   | 0x01 0x02 0x03 0x04 |
| 0x101 | 1 | 0     | 1   | 0x05 0x06 0x07 0x08 |
| 0x300 | 1 | 1     | 2   | 0xFA 0x04 0x0E 0x18 |
| 0x203 | 1 | 0     | 3   | 0x14 0x18 0x1C 0x20 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- Write C[4]: Miss, Write to dirty block, evict block 0x203
- A[4]: Hit
- Total: 9 misses, 37 hits

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x200 | 1 | 0     | 1   | 0x11 0x15 0x19 0x1D |
| 0x101 | 1 | 0     | 2   | 0x05 0x06 0x07 0x08 |
| 0x300 | 1 | 1     | 3   | 0xFA 0x04 0x0E 0x18 |
| 0x203 | 1 | 0     | 4   | 0x14 0x18 0x1C 0x20 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- Bt[4]: Miss, evict 0x300
  - Only at this point does main memory get updated
- Total: 10 misses, 37 hits, 1 write

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x200 | 1 | 0     | 3   | 0x11 0x15 0x19 0x1D |
| 0x101 | 1 | 0     | 1   | 0x05 0x06 0x07 0x08 |
| 0x300 | 1 | 1     | 4   | 0xFA 0x04 0x0E 0x18 |
| 0x301 | 1 | 1     | 2   | 0x6A 0x?? 0x?? 0x?? |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Caching Example: Write-Back Fully Associative with LRU

- A[5]-Write C[5]: 7 hits
- Compute C[6]: 1 miss, 8 hits
- Compute C[7]: 1 miss, 8 hits
- Total: 12 misses, 60 hits, 1 write

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x200 | 1 | 0     | 4   | 0x11 0x15 0x19 0x1D |
| 0x101 | 1 | 0     | 2   | 0x05 0x06 0x07 0x08 |
| 0x201 | 1 | 0     | 1   | 0x12 0x16 0x1A 0x1E |
| 0x301 | 1 | 1     | 3   | 0x6A 0x84 0x9E 0xB8 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|     |     |     |     |
|-----|-----|-----|-----|
| 250 | 260 | 270 | 280 |
|     |     |     |     |
|     |     |     |     |
|     |     |     |     |



# Caching Example: Write-Back Fully Associative with LRU

- Compute C[8]: 3 misses, 6 hits
- Compute C[9]: 1 miss, 8 hits, 1 write
- Compute C[10]: 1 miss, 8 hits
- Compute C[11]: 1 miss, 8 hits
- Total: 18 misses, 90 hits, 2 writes

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x202 | 1 | 0     | 4   | 0x13 0x17 0x1B 0x1F |
| 0x101 | 1 | 0     | 3   | 0x05 0x06 0x07 0x08 |
| 0x203 | 1 | 0     | 2   | 0x14 0x18 0x1C 0x20 |
| 0x301 | 1 | 1     | 1   | 0x6A 0x84 0x9E 0xB8 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|     |     |     |     |
|-----|-----|-----|-----|
| 250 | 260 | 270 | 280 |
|     |     |     |     |
|     |     |     |     |
|     |     |     |     |

# Caching Example: Write-Back Fully Associative with LRU

- Compute C[12]: 3 misses, 6 hits
- Compute C[13]: 1 miss, 8 hits, 1 write
- Compute C[14]: 1 miss, 8 hits
- Compute C[15]: 1 miss, 8 hits
- Total: 24 misses, 120 hits, 3 writes

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x102 | 1 | 0     | 3   | 0x09 0x0A 0x0B 0x0C |
| 0x202 | 1 | 0     | 4   | 0x13 0x17 0x1B 0x1F |
| 0x302 | 1 | 1     | 1   | 0xDA 0x04 0x2E 0x58 |
| 0x203 | 1 | 0     | 2   | 0x14 0x18 0x1C 0x20 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|     |     |     |     |
|-----|-----|-----|-----|
| 250 | 260 | 270 | 280 |
| 618 | 658 | 670 | 696 |
|     |     |     |     |
|     |     |     |     |

# Caching Example: Write-Back Fully Associative with LRU

- Final step: Flush the cache
  - Write back all dirty blocks
  - Invalidate all blocks
- Final Count: 24 misses, 120 hits, 4 writes

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x202 | 1 | 0     | 4   | 0x13 0x17 0x1B 0x1F |
| 0x103 | 1 | 0     | 3   | 0x0D 0x0E 0x0F 0x10 |
| 0x203 | 1 | 0     | 2   | 0x14 0x18 0x1C 0x20 |
| 0x303 | 1 | 1     | 1   | 0x4A 0x84 0xBE 0xF8 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|     |      |      |      |
|-----|------|------|------|
| 250 | 260  | 270  | 280  |
| 618 | 658  | 670  | 696  |
| 986 | 1028 | 1070 | 1112 |
|     |      |      |      |

# Caching Example: Write-Back Fully Associative with LRU

- Final Count: 24 misses, 120 hits, 4 writes
- 83% hit rate
  - Way better than the 50% we got before

| Tag   | V | Dirty | LRU | Data                |
|-------|---|-------|-----|---------------------|
| 0x202 | 0 | 0     | 4   | 0x13 0x17 0x1B 0x1F |
| 0x103 | 0 | 0     | 3   | 0x0D 0x0E 0x0F 0x10 |
| 0x203 | 0 | 0     | 2   | 0x14 0x18 0x1C 0x20 |
| 0x303 | 0 | 0     | 1   | 0x4A 0x84 0xBE 0xF8 |

A

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

Bt

|    |    |    |    |
|----|----|----|----|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

C

|      |      |      |      |
|------|------|------|------|
| 250  | 260  | 270  | 280  |
| 618  | 658  | 670  | 696  |
| 986  | 1028 | 1070 | 1112 |
| 1354 | 1412 | 1470 | 1528 |

# Agenda

- Intro
  - Binary Prefixes
  - Memory Hierarchy
- Intro to Caches
- Fully Associative (FA) Caches
- Matrix Multiply Example
- Eviction Policy
- Write-Back/Write-Through Caches
- **Analyzing Caches**

# Analyzing Cache Performance

- The **hit rate** of a cache is the percentage of accesses that result in a hit
- The **miss rate** of a cache is the percentage of accesses that result in a miss
- **Hit time** is how long it takes to check the cache. The **Miss Penalty** is how long it takes to access main memory after a miss.
  - If we get a cache hit, runtime is equal to hit time
  - If we get a cache miss, runtime is equal to hit time + miss penalty
  - If we didn't have a cache, we'd always take miss penalty time to access main memory
- Analogy: Hit time = how long to check the bookshelf, Miss penalty = how long to go to the library
- In the previous example, our hit rate was  $\frac{5}{6} = 83.33\%$ , so our miss rate was  $\frac{1}{6} = 16.66\%$
- The **Average Memory Access Time** (or AMAT) is the average amount of time it takes for one memory access, given a hit rate.

**In the matrix multiplication example, we had a 5/6 hit rate.  
If our hit time was 10 cycles and our miss penalty was 100  
cycles, what speedup was attained by our cache?**

2x

3x

3.75x

4x

5x

5.555...x

6x

To



0

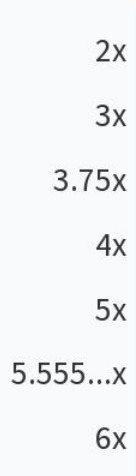
**In the matrix multiplication example, we had a  $5/6$  hit rate.  
If our hit time was 10 cycles and our miss penalty was 100  
cycles, what speedup was attained by our cache?**

2x  
3x  
3.75x  
4x  
5x  
5.555...x  
6x





**In the matrix multiplication example, we had a  $5/6$  hit rate.  
If our hit time was 10 cycles and our miss penalty was 100 cycles, what speedup was attained by our cache?**



# Analyzing Cache Performance

- $\frac{5}{6}$  of our memory accesses were hits, and therefore took 10 cycles each
- $\frac{1}{6}$  of our memory accesses were misses, and therefore took  $100+10=110$  cycles each
- 5 hits + 1 miss is 160 total cycles, so AMAT is  $160/6$  cycles
- Alternate formula:  $\text{AMAT} = \text{hit time} + (\text{miss rate} * \text{miss penalty})$ 
  - $\text{AMAT} = 10 + (\frac{1}{6} * 100) = 160/6$
- Our original runtime would have been 100 cycles per access
- Speedup is  $100 / (160/6) = 600/160 = 3.75x$

# Computing Hit Rate

- Computing AMAT can get tricky for certain programs
  - Main difficulty is in determining the hit rate of a program
- General Tips:
  - Generally speaking, the hit rate starts low, because the cache is cold
  - After warming up, the cache normally reaches a "steady state", where a pattern of hits and misses appears every iteration of some outer loop, until we completely exhaust the memory in a block
  - Once we start a new block, the cache acts cold again, and the cycle repeats.
  - Once you find this cycle, it's possible to use that to get the total hit rate.
  - This is a heuristic (not guaranteed), though, so you need to be rigorous with showing that your pattern will actually hold
- More practice: Homework